

1 - Exam facts, and how to attack a question

Read this page first on exam day. It is worth more than any single fact.

Questions 60	Time 120 min (2 min each)	Pass mark 720 / 1000	Scenarios 4, drawn from 6
D1 Agentic 27% (~16 q)	D2 Tools + MCP 18% (~11 q)	D3 Claude Code 20% (~12 q)	D4 Prompts / D5 Context 20% (~12 q) / 15% (~9 q)

Each question states **how many answers to select** - read that line. **No penalty for guessing: never leave one blank.** You will finish the first pass in 30-40 minutes - every mock has - which leaves ~80 minutes. **Do not leave.** Sweep twice before you stop: every *select two*, count your letters; every last sentence containing *never, not, except, instead*, re-read the stem. Those two sweeps were worth 5 marks on Mock 5 and cost nothing on Mock 6, which went 58/60 with both of them run.

READ THIS FIRST - the two checks that hold, and the one fact pair that does not

Say **how many letters you owe before you read the options.** Then, for two symptoms, ask "which symptom did this answer solve, and what solves the other?"

Underline never, not, except, instead in the last sentence. On an inverse stem the correct-sounding options are the wrong ones - the valid grounds will be sitting right there.

These two are working: Mock 6 ran 10 select-two and 4 inverse-direction questions and lost **nothing** to either (9/10, no blank, no single letter, 4/4 on direction), after Mock 5 lost 5 marks to exactly this. **Run them anyway - they are checks, not habits.**

What is still live is one sentence, not a habit: an incomplete answer that SAYS SO is correct; an incomplete answer that READS AS COMPLETE is the failure. That decides coverage gaps (declare them in the report, never in a log) and error status (partial results labelled partial are fine; a success with no results is not). Both on page 6.

The 30-second method

1	Read the last sentence of the question first. That is what is actually asked.
2	Note the signal words (page 2). They usually name the answer type.
3	Remove any answer describing a feature that does not exist .
4	Remove any answer blaming a part the question says is working correctly .
5	From what is left, choose the weakest fix that still meets the requirement .
6	Ask where the information should have been PRODUCED , not where the failure showed up.

Two mechanical triggers. (1) Domain 3: ask *what set of files does this content govern* - never which nouns the question mentions (page 3). (2) **Halt on repair verbs** - post-process, validate, reconcile, re-fetch, reverse (page 6).

Step 6, the cross-domain trap: the failure appears late, the fix belongs early. Field the source always has → **required**. Inconsistent dates or currency → **normalisation rules in the prompt**. Synthesis cannot judge a source → **subagents record dates, location, methodology**. The late fix is defensible engineering; that is why it is offered. Which component held the information when it was lost?

The strength ladder - choose the lowest level that meets the requirement

Level	Fix	Use when
1	Better tool descriptions	Model picks the wrong tool; descriptions are short or similar
2	Few-shot examples (2-4)	Behaviour is inconsistent in unclear cases
3	Explicit criteria in the prompt	Too many false alarms; current instruction is vague
4	Programmatic enforcement hooks, prerequisite gates, forced tool_choice	The rule must ALWAYS work
5	Separate classifier / ML system	Almost never the answer on this exam

Level 4 wins whenever the question mentions any of these

Money (refunds, payments, billing) - identity checks - policy limits or thresholds

The words **deterministic, guaranteed, must, or never**

Reason from the guide: prompt instructions have a **non-zero failure rate**. Sometimes is not acceptable when money is involved.

The five kinds of wrong answer

Too big a solution	Builds new systems before trying simple fixes. Signals: train a classifier, deploy a model, add a routing layer, build a service
Blames a working part	The question names the broken part. Any answer pointing elsewhere is wrong
Feature does not exist	Sounds confident, describes a flag or file that is not real (see page 3)
Solves another problem	Reasonable, but fixes something the question did not ask about
Moves work to people	Asks developers or customers to do more, or hides real problems

2 - Never-answers, signal words and file paths

Pure memorisation, and the cheapest points on the exam. This half has been clean 8 for 8.

Never the right answer

Self-reported confidence used to decide when to escalate
Sentiment analysis used to judge how difficult a case is
A bigger context window used to fix inconsistent quality or attention dilution
Voting across several runs to filter findings - this hides real bugs
Reading the model's text to decide when the loop stops - use <code>stop_reason</code>
A fixed iteration cap as the main stopping method (fine as a safety limit)
Hiding errors by returning empty results as success
Stopping the whole workflow because one subagent failed
Guessing when a tool returns several matches - ask for another identifier

Signal words in the question

Words in the question	What to choose
most effective first step	The cheap simple fix (level 1-3), not the big one
deterministic, guaranteed, must never	Programmatic enforcement (level 4)
the logs show the coordinator...	The cause is named. Fix that part
minimal descriptions	Improve the tool descriptions
blocking, developers wait	Real-time API, not batch
overnight, next morning, weekly	Batch API is fine
inconsistent results, contradictory feedback	Split into smaller focused passes
false positives, developer trust	Write explicit criteria
hangs, waiting for input	<code>-p / --print</code>
every developer when they clone the repo	Project scope: <code>.claude/...</code> in the repository
spread throughout the codebase	<code>.claude/rules/</code> with glob patterns
dozens of files, architectural decisions	Plan mode
single-file fix with a clear stack trace	Direct execution

File paths

Personal instructions - NOT shared	<code>~/.claude/CLAUDE.md</code>
Project instructions - shared	<code>.claude/CLAUDE.md</code> or <code>CLAUDE.md</code> in the root
Folder instructions	<code>CLAUDE.md</code> inside that folder
Rule files by topic	<code>.claude/rules/</code>
Shared slash commands	<code>.claude/commands/</code>
Personal slash commands	<code>~/.claude/commands/</code>
Shared skills	<code>.claude/skills/</code>
Personal skills	<code>~/.claude/skills/</code>
Shared MCP servers	<code>.mcp.json</code>
Personal MCP servers	<code>~/.claude.json</code>

Rule: anything starting with `~/` is personal and is NOT shared through version control.

3 - Domain 3: choosing the mechanism

The weakest domain again - 6/12 on 20 Aug, after 12/12 on a paper written from its own miss list. Every miss on 16, 17 and 20 August was on this page. None was a forgotten path.

Which mechanism holds which content - choose by the **GROUPING**, nothing else

The content is grouped by	Use	Because
nothing - it always applies	the root CLAUDE.md	Always loaded, no maintainer action
package or service , maintainer picks which shared documents apply	@import in that package's CLAUDE.md	Selective inclusion by reference, nothing duplicated
file type , wherever the file lives (**/*.test.tsx , **/*.stories.tsx)	.claude/rules/ with paths:	Loads only when a matching file is edited
one folder , everything in it, nothing outside	a CLAUDE.md inside that folder	Covers exactly that subtree
the moment someone asks	a skill	Must be invoked, so it can never always apply

Ask ONE question: what SET OF FILES does this content govern?

This is the single most expensive decision on the paper - four marks on 16 Aug, four more on 17 Aug, six on 20 Aug.

@import and .claude/rules/ have now been traded for each other inside ONE scenario three times (16, 17 and 20 Aug). Both mechanisms are known; each is being handed to the other's question. Recite the grouping sentence before reading the options, every time.

Do **not** read the grouping off the nouns in the question. "They exist in nine packages" is still a **file pattern** if the content governs *.stories.tsx. "Each lead decides which apply" is still **@import**, not a skill, because after the decision they govern ALL work there.

.claude/rules/ is the reflex answer and is correct **only** for a file pattern. **Skills** are never the answer for anything that must always apply.

Frontmatter keys - do not mix these two

SKILL.md (in .claude/skills/)	.claude/rules/ files
context: fork run in a separate context, keeping long output out of the main conversation	paths: glob patterns. The rule loads only when you edit a matching file
allowed-tools limit which tools the skill may use	Example: paths: ["terraform/**/*"] paths: ["**/*.test.tsx"]
argument-hint ask the developer for a missing parameter	

paths: is NOT a SKILL.md key. Mixing these two sets is the most common mistake in Domain 3.

Commands and flags

See which memory files are loaded	/memory
Reduce context in a long session	/compact
Continue a named session	--resume <name>
Branch from one shared analysis	fork_session
Non-interactive mode in CI - fixes hanging	-p or --print
Output a script can read	--output-format json
Force output to follow a schema	--json-schema
Noisy discovery without filling the context	Explore subagent

These DO NOT EXIST - they appear as wrong answers

--batch CLAUDE_HEADLESS=true --no-interactive

.claude/config.json with a commands list

Redirecting stdin from /dev/null as the "correct approach"

parallel: true in an AgentDefinition - parallel is several Task calls in one response

A committed .mcp.json "**disabled by default**", or a personal servers block in .claude/settings.json - personal servers live in **~/.claude.json**

Server-side session state on the API - it is **stateless**; send the whole history

4 - Domain 1: Agentic Architecture (27%)

The biggest domain. About 16 of the 60 questions.

The agentic loop

The loop is controlled by stop_reason . Nothing else.
"tool_use" -> run the tools, add results to history, repeat
"end_turn" -> stop
Continue on tool_use ONLY - never on "anything that is not end_turn". max_tokens, refusal and pause_turn all fail that test and would loop with nothing to execute
All seven API values: end_turn, max_tokens, stop_sequence, tool_use, pause_turn, refusal, model_context_window_exceeded. The exam tests the first pair
Every tool_use needs one tool_result with the same tool_use_id
Send all results from one Claude message back in ONE user message
Four wrong ways to stop the loop
1. Reading Claude's text for a phrase like "I have completed"
2. A loop counter as the MAIN stopping method (as a safety limit it is fine)
3. Checking whether the response contains text - text and tool calls appear together
4. An explicit task_complete tool - end_turn already signals completion

Subagents

Started with the Task tool
The coordinator's allowedTools must include "Task" - without it, no delegation is possible
Subagents receive NOTHING automatically. No history, no memory. Put it all in the prompt
Parallel = several Task calls in ONE response . Separate turns run sequentially. There is no flag, no setting, no parallel: true
fork_session = branches from ONE shared analysis, to compare approaches
All messages go through the coordinator . Subagents never talk to each other
AgentDefinition = each subagent's description, system prompt, and tool restrictions
A good coordinator chooses which subagents to invoke by the complexity of the request. Running five unnecessary subagents faster is still running five unnecessary subagents
The failure to remember
Coverage is incomplete BUT every subagent worked correctly
= the coordinator's task decomposition was too narrow
Any answer blaming a downstream agent is wrong.

Making a rule that always works

Programmatic enforcement	Prompt-based guidance
Hooks, prerequisite gates	System prompt, few-shot examples
Always works	Fails sometimes
Money, identity checks, policy limits, compliance	Style, preferences, judgement

The two hook types

PostToolUse	Changes tool RESULTS before the model reads them. Use for normalising different data formats (Unix time / ISO 8601 / numeric codes)
PreToolUse (the interception hook)	Blocks an outgoing tool CALL . Use for blocking refunds over \$500 and redirecting to escalation

Splitting up work

Ask first: **do I know the shape of the work before I start?**

Prompt chaining YES - shape known	The file list is known and the criteria are fixed. Large review = one pass per file + one pass across files
Dynamic decomposition NO - shape unknown	Structure unknown; each step depends on the last. Map first, then a plan that adapts. Example: add tests to a legacy codebase

"Audit", "large" and "comprehensive" do **NOT** decide this. 900 files, nobody knows where the code lives = **dynamic**. 22 files, known list = **chaining**. Same size, opposite answers.

A bigger context window does **NOT** fix attention dilution.

Sessions

A few known files changed	--resume, and name the changed files
Earlier tool results are mostly stale	Start a NEW session with a written summary

The question is **HOW MUCH** is stale, not whether anything is stale.

5 - Domain 2: Tool Design and MCP (18%)

About 11 of the 60 questions.

Tool descriptions - the core fact

Tool descriptions are how the model chooses a tool. Improve them first .
A description must contain: inputs , example queries , edge cases , and when to use it instead of a similar tool .
Overlapping tools -> rename and rewrite (analyze_content -> extract_web_results)
One tool doing three jobs -> split it, with defined inputs and outputs
Check the system prompt for words that create unwanted tool links
Improve MCP descriptions, or the agent prefers built-in Grep over a better MCP tool

Error responses

Category	Example	Agent should
transient	Timeout, service down	Retry
validation	Bad input	Fix the input, retry
business	Breaks a policy	Explain to the user. Do NOT retry
permission	Not allowed	Escalate or explain

Every error returns: errorCategory + isRetryable + a readable message
Business rules also need isRetryable: false and a customer-friendly explanation
An access failure is NOT an empty result . A timeout may need a retry; zero matches is a success
Subagents fix transient failures themselves . They only report what they cannot fix
When reporting: failure type + what was tried + partial results + alternatives
business vs permission - a policy limit exceeded is business (input and system are both fine, a RULE says no). The caller lacking access is permission

tool_choice

auto	May reply with TEXT instead of calling a tool. Never the fix when it returns text
any	Must call a tool, chooses which. Use when the document type is unknown
{"type":"tool","name":..."}	Must call THAT tool. Use to force extract_metadata to run first
none	Cannot use tools

auto may talk - any must act - forced picks the actor.

Tool distribution

18 tools is bad. 4-5 is good. More tools = worse selection
An agent with tools outside its job will misuse them
Give a small cross-role tool for a common need (verify_fact for the synthesis agent)
Replace general tools with limited ones (fetch_url -> load_document)

MCP configuration

Shared team servers	.mcp.json (project, version-controlled)
Personal / experimental	~/.claude.json (user)
Keep secrets out of the repo	\${GITHUB_TOKEN} environment variable expansion
MCP resources	Content catalogues: issue lists, doc structures, DB schemas. They stop the agent making exploratory calls
Standard integration (Jira)	Use a community server. Build custom only for your own workflows

Built-in tools

Grep	Searches INSIDE files - function callers, error messages, imports
Glob	Matches FILE PATHS - **/*.test.tsx
Edit fails on unclear text	Use Read + Write instead
Exploring a codebase	Grep to find entry points, then Read to follow imports. Do not read everything

6 - Domain 4: Prompts and Structured Output (20%)

Your weakest domain on the day-1 test. Read this page every day.

Schema design

Required or nullable? Ask: does the source ALWAYS contain this value? Source may NOT contain it → nullable , or a required field makes the model INVENT one. Source ALWAYS contains it → required , so the field cannot be OMITTED . Never "nullable + reject null" - that fails documents and still does not stop fabrication
Enum + "other" + a detail string, for categories that may grow
Enum value "unclear" for ambiguous cases
Tool use with a JSON schema is THE way to get structured output. Prompt asks for JSON in the response text and some replies will not parse → declare the schema as a tool input schema and return the record through tool use . Not more examples of well-formed JSON, not a lenient parser, not a lower temperature (<i>missed 20 Aug</i>)
Tool use + schema removes SYNTAX errors. It does NOT remove SEMANTIC errors
calculated_total next to stated_total - catches totals that do not add up
conflict_detected - boolean for contradictory source data
detected_pattern - to analyse which findings developers dismiss
The most expensive line on this card - missed FOUR times, always the same distractor Source states dates, currency or quantities in several formats → format normalisation rules in the PROMPT , next to the strict schema. The schema fixes the TYPE. The prompt fixes the FORMAT. A post-processing layer is the wrong answer: it needs a branch per variant and breaks on the first layout nobody anticipated. Same for a "validation check against the source". Halt on repair verbs - post-process, validate, reconcile, re-fetch, reverse, downstream. Then name the component that held the information when it was lost. That component is the answer.

Retries

Retry WORKS	Retry DOES NOT WORK
Wrong format (dates, currency)	The information is not in the document
Wrong output structure	It is only in a document you did not supply

A retry request contains: the **document** + the **failed extraction** + the **specific validation errors**.

Message Batches API

Cost	50% cheaper
Time	Up to 24 hours
Speed guarantee	NONE
Matching results	custom_id (never by position)
Multi-turn tool calling	Not supported
Good for	Overnight reports, weekly audits, nightly test generation
Bad for	Anything where a person is waiting (pre-merge, pre-commit)
Failures	Resubmit only the failures, found by custom_id. Split long documents
SLA maths	Worst case = gap between submissions + 24h . It must be under the SLA. 30-hour SLA → every 4 hours (28). 36-hour → every 8 (32)

Prompts and criteria

Specific categories work. "Be conservative" does not
Confidence-based filtering does not improve precision
A category with many false alarms damages trust in the good ones → turn it off while you fix it
Severity levels need concrete code examples for each level
Few-shot: 2 to 4 examples , aimed at the unclear cases, not the obvious ones
Few-shot lets the model generalise to new situations, not just copy
A prose description giving inconsistent results → 2-3 concrete input/output examples . The guide's most effective way to show an expected transformation. Not a skill
Unfamiliar domain, you do not know what to specify → the INTERVIEW pattern . Let Claude ask the questions first, so it raises invalidation, failure modes, the things you forgot. Not plan mode - plan mode designs against a specification you do not yet have
Problems that affect each other → ONE detailed message. Independent problems → one at a time
In CI, CLAUDE.md is how Claude Code learns the project : testing standards, fixture conventions, review criteria. Not appended to the prompt by the pipeline script each run.
Duplicate test suggestions → supply the existing test files

Review architecture

Self-review is weak: the model remembers its own reasoning → use an independent instance
Large review = one pass per file + one separate pass across files
A bigger context window does NOT fix attention dilution
Voting across runs hides real bugs
Confidence IS allowed next to each finding, to route reviewer attention

7 - Domain 5: Context and Reliability (15%)

Only 15%, but it appears in four of the six exam scenarios.

Three context problems and their fixes

Problem	Fix
Progressive summarization loses numbers, dates, and what the customer expected	A " case facts " block in every prompt, OUTSIDE the summarised history
Lost in the middle - the model handles the start and end well, misses the middle	Key findings at the BEGINNING . Clear section headings for the details
Tool results fill the context - 40+ fields when only 5 matter	Trim to the useful fields BEFORE they enter the context

Escalation

Escalate when	NEVER escalate based on
The customer asks for a human	Sentiment - it does not predict difficulty
There is a policy gap (not just a hard case)	The model's own confidence score
No progress is possible	

The trap - these two look the same
Customer explicitly asks for a human -> escalate IMMEDIATELY , do not investigate first
Customer is frustrated but you can help -> offer help, escalate only if they ask again
Several matching customers -> ask for another identifier . Never guess.

Errors between agents

Report four things: failure type + what was tried + partial results + alternatives
Never: a generic message like "search unavailable"
Never: hiding the error by returning empty results as success
Never: stopping the whole workflow because one agent failed
Never: treating an access failure and a valid empty result as the same thing
Partial results, LABELLED as partial, are always correct. A success with an empty result set when the lookup failed is always wrong - that is hiding the error as success. The tell is not how much data comes back, it is whether the status is honest

Coverage gaps - missed three times, three different wrong answers
The output must state which findings are well supported and which topics have gaps - in the report the reader sees, next to the findings.
A log line, a ticket, a monitoring alert or a note to the pipeline team are all the same miss. So are "retry until the sources respond" and "summarise more aggressively": a gap is declared , never removed.
Ask: would the person reading this report know that two sources answered where three were expected?
Same sentence as the error status above: an incomplete answer that says so is correct; an incomplete answer that reads as complete is the failure.

Four mechanisms that all sound like "keep the context clean". They are NOT interchangeable

The problem	The one mechanism
Noisy discovery whose output has no value once the answer exists	an Explore subagent - keeps it out in the first place
A live session is nearly full and you want to carry on in it	/compact
A long multi-agent job crashed and the work was lost	MANIFESTS - each agent writes state to a known location, the coordinator loads it on resume
The model talks about " typical patterns " instead of the specific classes it found (context degradation, nothing crashed)	Scratchpad files - key findings written down as they are established

Missed twice, with a **different** wrong answer each time (/compact, then scratchpads), which is the signal for a fact that is absent rather than out-competed. **Crashed = manifest**. Nothing else recovers lost work.

Human review and confidence

97% average accuracy can hide one broken document type
Analyse accuracy by document type and by field before reducing human review
Stratified random sampling of HIGH-confidence extractions - that is the group you are about to stop checking
Confidence is usable only when calibrated with labelled validation sets , and only for routing review
Route to a human: low confidence, or the source is ambiguous or contradicts itself

Sources and provenance

Attribution is lost during summarisation , when claim-source links are not preserved
Subagents output claim-source mappings : source URL, document name, relevant excerpt
Subagents must include publication dates , or old and new numbers look like a contradiction
Also source location and methodology - nobody downstream can recover these. Never "weigh credibility" or "re-fetch at synthesis time": provenance travels as structure
Two credible sources disagree -> record BOTH with their sources . Never pick one
Separate well-established findings from contested ones in the report
Format by content type: financial data = tables , news = prose , technical = lists